

基于 C++ 的低延迟语音交互系统 · 面试准备资料

适用范围：当前这一版项目（VOICE / LLM / TTS 三模块，ASR → LLM → TTS 低延迟闭环）。文档包含三部分：- 0. 简历术语速背卡 - A. 项目自我介绍口播稿 - B. 面试问答清单

0. 简历术语速背卡

《低延迟语音对话闭环》

- 一句话定义：从我开口说话到机器开口回话，整条路一气呵成不卡顿。
- 在本项目里指什么：麦克风采到的声音被识别成文字，文字交给大模型，大模型回的话再实时合成播放出来，整条链路自动跑完一轮。
- 面试可直接背诵版：我做的就是个端到端语音对话闭环，从用户说话开始，到机器把回答念出来，全程自动衔接，没有手动触发的环节，目标是把这一整轮时间压到很短。

《VOICE / LLM / TTS 三模块》

- 一句话定义：把整套系统拆成“耳朵 + 大脑 + 嘴巴”三个独立伙计。
- 在本项目里指什么：VOICE 负责听和总调度，LLM 负责想该说什么，TTS 负责把文字念出来，三个进程各管一摊。
- 面试可直接背诵版：我把系统拆成 VOICE、LLM、TTS 三个独立模块，分别对应听、想、说，每个模块都是一个独立进程，互相之间通过消息通信，这样职责清楚，调试和替换都很方便。

《ASR》

- 一句话定义：把人的声音翻译成文字的那道工序。
- 在本项目里指什么：在 VOICE 模块里跑，麦克风采到一段音频之后转写成一句中文文本，再交给后面的大脑。
- 面试可直接背诵版：ASR 就是语音转文字，我把它放在 VOICE 模块里，配合一个端点检测，用户说完一句之后自动截断、自动转写，然后这句文字就是后面流程的输入。

《端侧 / 云端》

- 一句话定义：在自己机器上跑叫端侧，在远程服务器跑叫云端。

- 在本项目里指什么：ASR 和 TTS 都在本机跑，LLM 走云端流式接口，三者拼成一个完整的对话。
- 面试可直接背诵版：识别和合成我都放在端侧，这样能省网络往返、还能保护语音原始数据；大模型放在云端，借助云端的大参数模型让回答质量更好，本地只负责跟它流式通话。

《ZeroMQ 混合通信模型》

- 一句话定义：用同一套消息库，但在不同环节挑不同的"说话方式"。
- 在本项目里指什么：进程之间有的环节需要一问一答，有的环节只需要源源不断单向送，我针对每一段链路挑了最合适的那种模式。
- 面试可直接背诵版：我用 ZeroMQ 做进程间通信，但没全用一种模式，而是按场景挑：要握手的地方用一问一答，要持续往下游推数据的地方用单向推送，这样每一段的语义都很明确。

《REQ/REP》

- 一句话定义：像对讲机点单，一句一句严格轮流，问完才能再问。
- 在本项目里指什么：VOICE 把一句识别文本递给 LLM 求一个 ack，TTS 播完最后一句也通过这条通道告诉 VOICE 可以开新一轮。
- 面试可直接背诵版：REQ/REP 就是必须严格一问一答、交替进行，问出去之前不能再问。我用它来做对话握手和"我播完了"这种状态确认，因为这种事情天然就是请求响应语义。

《PUSH/PULL》

- 一句话定义：像传送带，上游把货往上一放就走，下游谁有空就接。
- 在本项目里指什么：LLM 切出来的一句一句文本通过这条通道连续投给 TTS，LLM 不用等 TTS 任何回话。
- 面试可直接背诵版：PUSH/PULL 是上游只管发、下游只管收，天然支持多对一的负载均衡。我用它来把切好的句子源源不断推给 TTS，这一段我不需要确认，只需要尽量快地把句子送过去。

《一轮对话握手》

- 一句话定义：用户说完一句话之后，正式启动这一轮处理的那次确认。
- 在本项目里指什么：VOICE 把转写好的文本递给 LLM，LLM 立刻回一个简短 ack，让 VOICE 这边不被阻塞，可以去做别的事。
- 面试可直接背诵版：用户说完话之后，我会让 VOICE 跟 LLM 做一次很短的握手，LLM 收到文本就立刻回个 ack，这样 VOICE 不会卡在那儿等大模型生成完，整条流水线才能并行起来。

《播放结束反馈》

- 一句话定义：TTS 把这一轮最后一句念完之后，回头喊一声"我念完了"。
- 在本项目里指什么：TTS 播完带结束标记的那一帧之后，通过 REQ/REP 通道告诉 VOICE 可以重新打开麦克风、开始下一轮。
- 面试可直接背诵版：TTS 播完最后一句之后会回一条结束反馈给 VOICE，VOICE 收到才会重新打开麦克风，这样避免扬声器还在念回答的时候 ASR 把自己的声音误识别成用户输入。

《流式分句单向推送》

- 一句话定义：边切边发，发出去就不等回信，下一句继续发。
- 在本项目里指什么：LLM 一边接收云端的增量回复，一边切成一句一句，从切出来的那一刻就立刻推给 TTS，整条方向是单向的。
- 面试可直接背诵版：我让 LLM 一边收 token 一边切句，每切出一句立刻通过单向通道推给 TTS，不等 TTS 任何反馈，这样首句到达 TTS 的时间就跟整段生成完全解耦了。

《云端流式对话 API》

- 一句话定义：云上大模型的一种说话方式，不憋一大段，而是边想边吐字。
- 在本项目里指什么：LLM 模块向云端发起一次对话请求，云端通过 HTTP 长连接不停回 token，本地一边收一边处理。
- 面试可直接背诵版：我用的是云端流式对话接口，请求体是 messages 数组，符合 OpenAI 兼容协议，给多轮历史预留了扩展位；调用方式是流式的，云端边生成边把 token 推下来，我这边可以边收边切句。

《增量回调》

- 一句话定义：每来一小段新内容，就喊我一声让我处理一下。
- 在本项目里指什么：HTTP 流每收到一小段新文字，就回调一次切句器，把这段拼到累计缓冲里再判断要不要切句。
- 面试可直接背诵版：增量回调就是每来一小块新数据，框架就回调我一次，我在回调里把新内容追加到累计缓冲，然后扫一遍标点决定要不要切句，整个切句逻辑是事件驱动的，不是定时轮询的。

《中文强弱标点切句策略》

- 一句话定义：用中文的句号叹号当硬切，逗号顿号当软切，按情况下刀。

- 在本项目里指什么：句号、问号、叹号、分号、换行算强标点优先切；逗号、顿号、冒号算弱标点，配合首句最小字数、最大句长这些阈值再决定切不切。
- 面试可直接背诵版：我把中文标点分成强弱两档，强标点直接切，弱标点要配合最小字数、最大句长一起判断，避免出现"嗯，"这种碎片，也避免一直不切句导致首句迟迟出不来。

《句子片段实时下发》

- 一句话定义：哪怕只切出半句话，只要满足条件，就先送下去合成。
- 在本项目里指什么：切句器一旦判定可以成句，就立刻把这一段文字推给 TTS 排队合成，不等大模型整段说完。
- 面试可直接背诵版：我不会等大模型把整段话生成完再送 TTS，而是切一句送一句，TTS 那边其实是边接收、边合成、边播放的，这就是为什么用户感觉机器很快开口。

《token》

- 一句话定义：大模型一次只吐一小块文字，那一小块就是一个 token。
- 在本项目里指什么：云端流式接口每次回来的就是 token 级的增量，我累加起来再做切句。
- 面试可直接背诵版：token 就是大模型生成的最小文字单位，一次只吐一小段。我这边是流式接收 token，几十个 token 累积出第一个完整短句的时候，首句就已经送去合成了，所以首音时延才能压下来。

《文本队列 / 音频队列》

- 一句话定义：两条传送带，一条放等着念的句子，一条放念好的音频。
- 在本项目里指什么：LLM 推过来的句子先进文本队列，合成线程从这边取出来合成成 PCM，结果再放进音频队列，播放线程从音频队列取出来播放。
- 面试可直接背诵版：TTS 内部有两条队列，文本队列是合成线程的进货口，音频队列是播放线程的进货口；这样合成和播放完全解耦，前面合成快慢不影响后面的连续播放节奏。

《双线程异步模型》

- 一句话定义：一个线程专门做菜，一个线程专门上菜，互不干扰。
- 在本项目里指什么：合成线程当生产者从文本队列拿句子合成，播放线程当消费者从音频队列拿音频播放，两边靠互斥锁加阻塞等待加唤醒来同步。
- 面试可直接背诵版：TTS 我做成了双线程异步：合成线程是生产者，播放线程是消费者，中间用队列加阻塞等待加唤醒来同步；这样合成时间长一点也不会卡住播放，播放也不会反过来拖住合成。

《流式切句前置投递》

- 一句话定义：还没等大模型把整段话讲完，就把开头那截先送出去。
- 在本项目里指什么：LLM 切出第一句话的瞬间就经由单向通道推给 TTS 排队合成，让合成尽早开始、播放尽早开始。
- 面试可直接背诵版：我做了一个前置投递的设计：LLM 切出第一句的那一刻就立刻投给 TTS，让 TTS 在大模型还在生成后半段的时候就已经在合成、在播放，整条链路是流水线，不是串行的。

《前移首句发声时机》

- 一句话定义：让机器更早开口，而不是把整段答案憋出来再念。
- 在本项目里指什么：靠流式切句、单向推送、双队列异步合成，把"用户说完话"到"机器念出第一个字"这段时间显著压低。
- 面试可直接背诵版：我的优化目标就是前移首句发声时机，让用户尽早听到机器开口。手段是流式接收、流式切句、立刻投递、合成播放再并行，所以首句不需要等整段答案，只需要等前几十个 token。

《整轮语音交互闭环 4 s 以内》

- 一句话定义：从用户说完话到机器把这一轮回答念完，整轮压在 4 秒里。
 - 在本项目里指什么：把识别、握手、流式生成、切句、合成、播放串成流水线，整轮端到端控制在 4 秒以内。
 - 面试可直接背诵版：我把整轮闭环控制在 4 秒以内，这个 4 秒是从用户说完话开始算，到机器把这一轮回答念完为止，包含识别、握手、流式生成、切句、合成、播放所有环节，靠的就是流式加流水线，不是靠堆硬件。
-

A. 项目自我介绍口播稿

这个项目我做的是一个基于 C++ 的低延迟语音交互系统，整体目标就一句话：用户说一句话，机器尽快开口回答，整轮闭环我压在 4 秒以内。我把系统拆成三个独立的模块，也是三个独立进程：VOICE 负责听和总调度，LLM 负责想这一句要说什么，TTS 负责把答案念出来。

形态上，ASR 和 TTS 我都放在端侧，因为这两段对延迟和隐私都比较敏感，本地跑最稳；LLM 我接的是云端流式对话接口，借助云端模型的能力让回答质量更好。三个进程之间我用 ZeroMQ 做通信，但是没有全用同一种模式，我是按语义挑的：VOICE 跟 LLM 之间是一问一答的握手，VOICE 跟 TTS 之间也是一问一答，用来告诉对方“我这一轮播完了，可以开新一轮”；而 LLM 到 TTS 这一段，是单向推送，LLM 切一句推一句，不等任何回信，专门用来跑生成到合成这条主干。

LLM 模块这边，我是 HTTP 流式接收云端 token，每来一小段就在增量回调里追加到累计缓冲，然后用一个中文标点切句器去判断要不要切句。我把中文标点分成强弱两档，强标点直接成句，弱标点要配合首句最小字数、普通句最小字数、最大句长这些阈值一起判断，目的是让第一句尽早成型，但是又不会出现碎片化的半句话。一旦切出一句，立刻通过单向通道投递给 TTS，TTS 不需要等下一句。

TTS 模块我做成了双线程异步模型：合成线程是生产者，从文本队列里取句子做端侧合成；播放线程是消费者，从音频队列里取已经合成好的音频去播放。两边用阻塞等待加唤醒同步，互不干扰。每一轮我都拿一个结束哨兵贯穿整条链路，TTS 播到带结束标志的那一帧时，再通过握手通道告诉 VOICE 可以重新开麦。整套设计的结果就是：首句在收到前几十个 token 时就已经开始合成、开始播放，整轮端到端控制在 4 秒以内。

B. 面试问答清单

维度 1 · 项目背景与架构整体设计

Q1. (项目背景与架构整体设计) 你这个项目最初想解决的问题是什么？为什么要把系统拆成 VOICE、LLM、TTS 三个独立模块？

考察意图：看候选人有没有真正想过架构的"分"和"合"，而不是照搬别人结构。回答要点：- 我想做的就是能流畅说话的语音助手，核心痛点是"机器要尽快开口"，所以整套设计都是围着首句时延和整轮闭环时长来的。- 我把听、想、说拆成三个独立进程，是因为这三件事的资源画像完全不一样：听这边吃麦克风和 ASR，想这边吃网络 IO 和 token 流，说这边吃 CPU 算合成还要独占音频设备，放在一个进程里互相干扰太严重。- 拆开之后好处是清晰，任何一个模块崩了不会带崩别人，我可以单独拉起来调试，比如单测 TTS 的时候直接 mock 一个 LLM 推流就行。- 我也考虑过另一种思路，就是合在一个进程里靠线程拆分，看起来开销更小，但我没选，原因是调试和故障隔离都麻烦，而且我希望以后能把 LLM 进程换部署位置而不动其他模块。

Q2. (项目背景与架构整体设计) 你这个系统的整体数据流是怎么走的？请把"用户开口"到"机器念完"这一整条路串一下。

考察意图：看候选人能不能讲清端到端链路，并把异步部分讲明白。回答要点：- 用户开口之后，VOICE 这边用端点检测判断这一句说完了，把音频转写成中文文本。- VOICE 把这句文本通过一问一答的通道递给 LLM，LLM 收到立刻回一个 ack，VOICE 这边不会被卡。- LLM 同时向云端流式接口发起请求，每收到一小段增量就累积、判断切句，一旦能成句就通过单向通道推给 TTS。- TTS 这边合成线程从文本队列取句子合成，播放线程从音频队列取音频播放，两条线异步推进。- 这一轮的最后一句带结束标记，TTS 播到那一帧之后通过握手通道反馈 VOICE，VOICE 才会重新打开麦克风开下一轮。

Q3. (项目背景与架构整体设计) 端侧和云端是怎么分工的？为什么 ASR、TTS 你都放本地，只有 LLM 走云端？

考察意图：看候选人能不能基于延迟、隐私、算力做出合理的边界划分。回答要点：- ASR 我放端侧，主要因为它直接吃用户原始音频，走云端意味着每句话都要把音频往外送，延迟和隐私都不划算，本地用流式 ASR 加端点检测够用。- TTS 也放端侧，理由类似：合成结果要立刻播放，走云端会多一段网络回传时间，而且会受网络抖动影响连续播放。- LLM 我让它走云端，是因为大模型的参数体量本地跑不动，云端模型能力强、可替换，我只需要按统一的对话接口规范调它就行。- 我也想

过把 LLM 换成本地小模型来彻底端侧化，这个我没选，是因为我项目主线想验证的是"低延迟流式链路"本身，本地小模型质量牺牲太多。

维度 2 · ZeroMQ 基础与本项目通信模式选型

Q4. (ZeroMQ 基础与本项目通信模式选型) ZeroMQ 是什么？它跟普通 socket 比有什么不一样？

考察意图：HR 友好题，看候选人能不能用大白话解释中间件，不要堆术语。回答要点：- ZeroMQ 在我这这就是一个跨进程发消息的库，我可以把它理解成升级版的"信箱"，每条消息天然带边界，不像裸 socket 还要自己拼字节。- 它最大的特点是预置了好几种通信范式，比如一问一答、单向推送、广播等等，我只要按场景挑一种，不需要自己造协议状态。- 它还内置了重连和发送缓冲，进程之间起停的顺序不那么敏感，对本地多进程开发非常友好。- 我选它的原因就是这两点：消息边界天然干净，通信范式现成可用，省得我自己重复造轮子。

Q5. (ZeroMQ 基础与本项目通信模式选型) 你为什么不全用 REQ/REP？为什么 LLM 到 TTS 这一段要单独换成 PUSH/PULL？

考察意图：考察候选人能不能针对场景挑选模式，并讲出 trade-off。回答要点：- REQ/REP 的语义是必须严格一问一答、交替进行，问出去之前不能再问，这种语义很适合握手和状态确认，比如"这一句我收到了""我这轮播完了"。- 但 LLM 到 TTS 这一段是流式生产，我可能一秒钟切出好几句，如果每句都要等 TTS 回话，我就被 TTS 的合成速度拖死了，首句时延也压不下来。- 所以这一段我换成 PUSH/PULL，上游只管发、下游只管收，发完就走，下游有多少消费能力都自己消化，这天然符合"流式生产、异步消费"的语义。- 我也想过让 LLM 推完一句就同步等 TTS 回一个 ack，但我没选，理由就是这会把流水线退化成串行，整套异步推送的好处就没了。

Q6. (ZeroMQ 基础与本项目通信模式选型) 那为什么不直接用 Pub/Sub 广播？

考察意图：考察对几种 ZMQ 范式语义的区分。回答要点：- Pub/Sub 是广播语义，上游不在乎下游在不在、收没收到，订阅者来晚了就丢消息，这跟我场景不匹配。- 我这条 LLM 到 TTS 的链路是"切出来的每一句都必须被 TTS 收到并合成"，不能丢，丢一句用户就听不到那句话。- PUSH/PULL 就比 Pub/Sub 合适：上游有发送缓冲，下游没起也能先攒着，下游一上来就能拿到数据，整体不丢消息。- 而且 Pub/Sub 在一对一场景下其实没带来好处，反而要处理订阅时机问题，我不需要这种复杂度。

Q7. (ZeroMQ 基础与本项目通信模式选型) 你为什么要自研一个 zmq-comm-kit ? 它解决了什么具体痛点 ?

考察意图：考察是否会做适度的抽象封装，而不是过度设计或者完全裸用。回答要点：- 我直接用裸绑定的时候发现每个进程都要重复写一堆样板：建 context、绑端口、设超时、捕获异常、退出时清理资源，写多了既啰嗦又容易写错。- 所以我抽了一个基类把这些通用动作收口，包括 RAII 风格的资源管理和统一的异常类型；再派生出 Server、Client、Pusher、Puller 四种角色，每种角色只暴露它该有的方法。- 这样我在 voice、llm、tts 三个进程里写通信代码都是同一套语义，新增一种通道也只是派生一下，迭代很快。- 我也想过完全裸用绑定不封装，看上去更"贴近底层"，但我没选，因为三个进程会把同样的样板写三遍，后续改超时策略要改三个地方，不划算。

维度 3 · 云端 LLM 流式接入

Q8. (云端 LLM 流式接入) 你说的"增量回调"具体是怎么工作的？为什么不等整段生成完再处理？

考察意图：考察候选人对流式处理的真实理解。回答要点：- 我跟云端是 HTTP 流式接收，连接打开之后，云端会一小段一小段地把新生成的 token 推下来，每来一小段就触发一次我注册的回调。- 我在回调里做的事情很简单：把这次的新内容追加到累计缓冲，然后让切句器扫一下缓冲，看能不能切出一句来。- 能切就立刻把这一句通过单向通道投给 TTS，缓冲里只留剩下的那截没成句的部分，等下一次回调继续追加。- 我不等整段是因为等整段就意味着用户必须等完整答案生成完才能听到第一个字，那首句时延就直接被大模型的总生成时间吃掉了，整轮闭环 4 秒以内就做不到。

Q9. (云端 LLM 流式接入) 增量数据可能在一个标点中间被截断，半句粘包你怎么处理？

考察意图：考察对流式协议边界问题的真实经验。回答要点：- 我做的是"累计缓冲 + 标点扫描"的模式：每次回调都只往缓冲里追加，不假设这次回来的就是完整一句。- 切句判断完全基于扫描缓冲里的中文标点，扫不到强弱标点就什么都不做，等下一次回调把后半部分拼上再扫一次。- 我还专门处理了开头噪声标点的情况，比如缓冲一开头就是逗号或者句号，那直接跳过，避免切出一个空句子。- 还有一种情况是长时间不出现强标点，我会用最大句长兜底，强行切一下，避免缓冲无限增长、TTS 一直没东西可念。

Q10. (云端 LLM 流式接入) 你说有三级超时, 是哪三级? 为什么要分这么细?

考察意图: 考察候选人对真实网络异常的认识。回答要点: - 我做了三级: 连接超时、读空闲超时、单轮总超时。 - 连接超时管的是握手阶段卡住, 比如云端那边接入异常, 这种我希望几秒内就快速失败, 立刻换备选策略。 - 读空闲超时管的是连接建上了但中间长时间没有新 token 下来, 这种我也要让它快失败, 否则用户会一直听到空气。 - 单轮总超时是兜底, 哪怕 token 在缓慢地往下, 整轮加起来超过阈值我也强制结束, 因为对用户来说 4 秒以上的沉默就已经是事故了。 - 我也想过只做一个大超时, 简单但是不够细, 分级的好处是可以针对不同失败模式给不同处理。

Q11. (云端 LLM 流式接入) 你怎么实现"新一轮请求触发当前请求 cancel"的? 为什么要这么设计?

考察意图: 考察候选人是否真的在流式接入里实现过"打断"语义。回答要点: - 我给当前在跑的请求都关联一个可取消的句柄, 一旦 VOICE 把新一轮对话握手发过来, LLM 这边就先去把上一轮那个句柄置为取消, 立刻断掉连接, 回调里也会判断取消位直接返回。 - 这样之后切出来的所有句子都不会再投到 TTS 那条单向通道, 避免上一轮的尾巴跟新一轮的开头混在一起。 - 我也想过让上一轮自然跑完再开新一轮, 听起来更"完整", 但我没选, 因为用户体验上插话才更重要, 如果用户已经说了第二句, 他不可能想再听完上一句的回答。 - 配合 cancel, 我在 TTS 那边也准备了停止播放的语义, 整体打断就完整了。

Q12. (云端 LLM 流式接入) 多轮对话历史和 API Key 你是怎么管的?

考察意图: 考察候选人有没有为生产环境做基本的工程化考虑。回答要点: - 我给云端发的请求体就是 messages 数组, 符合 OpenAI 兼容协议, 结构上已经给多轮历史预留了位置, 当前默认是 system + user 两条。 - 要扩展多轮历史就是在 messages 里维护一段滚动窗口, 再加一个截断策略防止上下文超长, 这条路是直接通的, 不需要改协议。 - API Key 我只通过环境变量加载, 配置文件里绝对不写, 日志里凡是涉及 key 的字段都会脱敏。 - 模型名我也走白名单校验, 配置写错或者被人改成不允许的模型, 进程启动阶段就直接拒绝, 不会到运行时才出问题。

维度 4 · 中文流式切句策略与首音延迟优化

Q13. (中文流式切句策略与首音延迟优化) 你切句为什么是按标点切, 不直接按 token 数切?

考察意图: 考察候选人在策略层面是不是真的想过 trade-off。回答要点: - 按 token 数切实现简单, 但 TTS 念出来会很怪, 因为可能在词或者短语中间断开, 听感像机器卡壳。 - 中文标点天然就

是语义边界，按强弱标点切出来的句子，TTS 念出来就是正常的停顿节奏。 - 我也想过两种结合，比如"先按标点，没标点就按 token 数兜底"，这个其实我做了，就是最大句长策略，但主刀还是标点。 - 还有一个隐性好处是：按标点切，每一句的长度自适应，短句快出去、长句撑到位，跟人说话的节奏很接近。

Q14. (中文流式切句策略与首音延迟优化) 强弱标点和那些字数阈值是怎么设计的？为什么要设首句最小字数？

考察意图：考察候选人是否真的盯过首音体验。 回答要点： - 强标点是句号、问号、叹号、分号、换行这种语义完结的，扫到就立刻切。 - 弱标点是逗号、顿号、冒号，扫到不一定切，要看当前累计长度，因为弱标点切出来的可能只是个短语，太短了 TTS 念出来会一顿一顿。 - 首句我专门设了一个最小字数阈值，目的是避免第一句太短，比如"好的，"这种碎片直接被合成播出去，听感会很尴尬。 - 同时我又不希望首句拖太久，所以最小字数设得不大，目标是"够成一个完整短语就行"，整体在"听感自然"和"首音尽早"之间找平衡。

Q15. (中文流式切句策略与首音延迟优化) 大模型有时候会输出思考链或者 markdown 符号，你怎么避免这些被念出来？

考察意图：考察候选人有没有处理过真实大模型输出里的脏东西。 回答要点： - 我在切句器之前先过一层文本清洗，把 markdown 加粗符号、代码块围栏、列表项符号这种纯排版用的字符直接去掉，因为 TTS 念出来全是噪声。 - 思考链这种内容我是靠模型侧的约束 + 文本侧的开头识别一起处理：约束模型不要输出思考过程，万一漏出来我也能从开头特征段把那一截过滤掉。 - 我还会跳过开头的孤立标点，比如缓冲一开始就是一个换行或者逗号，这种直接吃掉，不算成句。 - 这些都是踩出来的，刚开始我没做清洗，TTS 直接把"加粗加粗加粗"念出来，体验非常崩。

维度 5 · TTS 双队列生产者-消费者并发模型

Q16. (TTS 双队列生产者-消费者并发模型) 你能用最直白的话解释一下你 TTS 里互斥锁和阻塞等待 + 唤醒是怎么配合的吗？

考察意图：HR 友好题，考察基础并发概念的口语化表达。 回答要点： - 我有两条队列，文本队列和音频队列，每条队列都有一把锁，谁要往里塞或者往外取，都先拿锁，避免两个人同时改导致数据乱掉。 - 取的时候，如果队列是空的，那这个取的人就阻塞等在那里，不会空转浪费 CPU。 - 塞东西的人塞完之后会喊一声，把阻塞等着的那个人唤醒，唤醒之后他重新拿锁、再取一次。 - 这一整套就是"锁保证不冲突、阻塞 + 唤醒保证不空转"，TTS 内部合成线程和播放线程就是靠这一套配合起来的。

Q17. (TTS 双队列生产者-消费者并发模型) 为什么要做双队列？合成和播放放一个线程串行做不是更简单吗？

考察意图：考察候选人能不能解释这种异步分离的真实价值。回答要点：- 串行做的话，合成一句的时间会直接卡住播放，播放期间合成也不能开始，整体节奏会一卡一顿。- 双队列之后，合成线程可以一直往前合成，把音频先攒在音频队列里，播放线程只管按节奏取出来播。- 这样合成稍微慢一点也不会立刻断音，因为音频队列里还有缓冲；同时播放在播第一句的时候，第二第三句已经在被合成，整体形成流水线。- 我也试过单线程串行，跑出来的效果就是首音之后会有明显间隙，因为每句之间都要花时间合成下一句，所以我才换成双队列。

Q18. (TTS 双队列生产者-消费者并发模型) 你说每轮用 "END" 哨兵贯穿整条链路，is_last 标志又是干嘛的？怎么保证不会越界播放？

考察意图：考察候选人是否真的设计过"轮次边界"。回答要点：- "END" 是文本层的轮次结束标志，LLM 切完最后一句之后会发一个 END 给 TTS，告诉它这一轮没下文了。- 合成线程从文本队列取到 END 的时候，不会把它合成成音频，而是会在前一个真正的音频片段上打一个 is_last 标志。- 播放线程播完带 is_last 的那一帧之后，才通过握手通道告诉 VOICE 这一轮真正结束了，VOICE 才会重新开麦。- 这样无论合成和播放的相对节奏怎么变，"这一轮播完了"这个事件都是从音频层冒出来的，不会出现 VOICE 提前开麦把自己刚念的话又录进去的情况。

维度 6 · 健壮性、可观测性与扩展性

Q19. (健壮性、可观测性与扩展性) TTS 播放的时候 ASR 不会把回声误识别成用户输入吗？你怎么避免？

考察意图：考察候选人对实际场景里"回声自识别"问题的处理。回答要点：- 我的核心做法是用握手把麦克风的开关跟 TTS 的播放严格串起来：TTS 一旦开始播这一轮回答，VOICE 那边的麦克风采集就处于关闭状态。- 等 TTS 播完带 is_last 的那一帧，通过握手通道反馈一次，VOICE 才把麦克风重新打开，开始监听下一句。- 这样在播放期间，ASR 根本拿不到音频输入，自然也就不可能把扬声器念出来的内容当成用户的新输入。- 我也考虑过用回声消除算法去做，那是另一条路，门槛更高、调参也复杂，对个人项目来说"放完再开麦"已经能很好地解决这个问题。

Q20. (健壮性、可观测性与扩展性) 三个进程任意一个挂了, 整套系统会怎么表现? 你做了什么兜底?

考察意图: 考察候选人对故障隔离和恢复的考虑。回答要点: - 因为是三个独立进程加消息通信, 任意一个挂掉不会带挂另外两个, 只会让消息暂时堵在通信层, 进程拉起来之后能接着收。 - LLM 这一段我专门做了流式无输出回退到一次非流式的兜底, 避免云端那边偶发卡顿就让用户听到一片寂静。 - 我还在 LLM 这边加了三级超时, 让每一种失败模式都能在固定时间内被识别, 不会出现进程还活着但是"死活不出声"的情况。 - 配置走 ini + 环境变量双通道, API Key 只走环境变量、模型走白名单校验, 这些是把配置错误造成的运行时崩溃前移到启动期的兜底措施。

Q21. (健壮性、可观测性与扩展性) 这套系统以后想扩展, 比如加多轮历史、加新的 TTS 引擎、跨机器部署, 分别要动哪些地方?

考察意图: 考察候选人架构上的"可扩展性意识"。回答要点: - 加多轮历史的话, 我在 LLM 模块里 messages 数组本身就给历史预留了位置, 只要加一个对话历史窗口管理 + 截断策略就行, 其他模块不用动。 - 换 TTS 引擎, 只要替换合成线程内部的合成实现, 文本队列、音频队列、is_last、END 这套接口完全不用改, 对外行为不变。 - 跨机器部署也很自然, 因为模块之间走的是 ZeroMQ, 只要把端口换成对应机器的地址就能跨机跑, 比如把 LLM 进程放到另一台机器上。 - 我设计的时候就刻意让模块之间只通过消息耦合, 不共享内存、不共享文件, 目的就是让这种扩展不会引发雪崩式的改动。